

Estándar de codificación para C# y .NET

Índice

1. Introducción	2
1.1. Lenguaje de desarrollo	2
1.2. Idioma del código fuente	2
1.3. Alcance del estándar	2
2. Propósito	3
2.1. Mantenibilidad	3
2.2. Legibilidad	3
2.3. Consistncia del código	3
2.4. Detección temprana de errores	3
2.5. Productividad del equipo	3
3. Reglas de nombrado	4
3.1. Variables	4
3.2. Campos	4
3.3. Constantes	5
3.4. Parámetros	5
3.5. Métodos	6
3.6. Clases	6
3.7. Interfaces	7
3.8. Enumeraciones	8
3.9. Propiedades	8
3.10. Eventos	10
3.11. Manejadores de eventos	10
3.12. Delegados	11
3.13. Namespaces	11
3.14. Archivos	11
3.15. Abreviaturas válidas	12
3.16. Prefijos y Sufijos	12
4. Estilos de código	14
4.1. Indentación	14
4.2. Longitud máxima de línea	14
4.3. Uso de espacios en blanco	15
4.4. Llaves	15
4.5. Paréntesis	16
5. Organización del código dentro de una clase	17
6. Prácticas de programación	18
6.1. Funciones Lambda	19
6.2. Uso de LINQ	22
7. Estructuras de control	25

7.1. if - else	25
7.2. If de una línea	25
7.3. Else If	26
7.4. Switch	27
7.5. Operadores ternarios	28
7.6. While	28
7.7. Do-While	29
7.8. For	29
8. Manejo de Excepciones	31
8.1. Bloques Try-Catch	31
9. Registro y depuración (bitácora)	33
9.1. Identificación de tipos de error	34
10. Comentarios	35
10.1. Comentarios de línea única	35
10.2. Comentarios de bloque	36
10.3. Comentarios de una línea en XAML	37
10.4. Comentarios de API pública	37
11. Seguridad	39
12. Pruebas	40
12.1. Nombrado de pruebas	40
13. Control de versiones	41
13.1. Sistema de control de versiones	41
13.2. Repositorio	41
13.3. Estructura de ramas	42
13.4. Commits	42
13.5. Pull Requests	43
13.6. Resolución de conflictos	44
14. Declaración de IA	45
14.1. Prompts	45
15. Bibliografía	47

Tecnologías para la Construcción de Software

Docente: Juan Carlos Pérez Arriaga

Estudiantes: Gabriel Antonio González López

Universidad Veracruzana, Facultad de Estadística e Informática, Ingeniería de Software

Chapter 1. Introducción

Este estándar está dirigido para estudiantes de Tecnologías para la Construcción de Software en la carrera de Ingeniería de Software, se recomienda usarlo para el desarrollo de los proyectos en clase y productos de software, con el objetivo de establecer una base para la construcción y lectura del código.

1.1. Lenguaje de desarrollo

Este estándar está pensado para aplicarse al lenguaje de C# como lenguaje principal del proyecto.

C# es un lenguaje de programación moderno, innovador, de código abierto, multiplataforma orientado a objetos.

1.2. Idioma del código fuente

El código fuente debe escribirse en inglés. Se tomó esta decisión para motivar a los estudiantes a desarrollar y mejorar su dominio del idioma, así como adaptarse al ambiente laboral general del mundo.

En cuanto al lenguaje de la GUI (Graphic User Interface) puede ser en cualquier idioma, los proyectos pueden incluir varios perfiles de idiomas.

1.3. Alcance del estándar

El estándar abarca buenas prácticas de programación (convenciones del lenguaje brindadas por Microsoft), su legibilidad, orden, espaciado y nombramiento, manejo de excepciones, registros y reglas de pruebas.

Chapter 2. Propósito

Este estándar tiene como propósito brindar especificaciones que ayuden a desarrollar un código seguro, mantenible y legible con C#. Así como brindar un seguimiento de parámetros definidos para contar con una guía clara y estructurada para el desarrollo de software apegada completamente a las siguientes pautas que hemos definido.

Se entiende como código seguro a aquel que no sea susceptible a inyecciones de código, acceso a datos sensibles no cifrados o sin una verificación de credenciales previa.

2.1. Mantenibilidad

Se invita al desarrollador a seguir buenas prácticas de diseño y construcción para que el producto resultante sea mantenible y el esfuerzo extra necesario para hacer un nuevo *release* sea el mínimo necesario.

2.2. Legibilidad

Al seguir ciertas recomendaciones de estilo de código se aumenta la legibilidad del código escrito, lo que facilita el encontrar errores de sintaxis, de lógica o refactorizar disminuyendo el tiempo necesario de inversión.

2.3. Consistncia del código

Cuando el código que se escribe está estandarizado y sigue un estilo constante las personas que lo lean o trabajen en el requieren menos tiempo para entenderlo y/o modificarlo.

2.4. Detección temprana de errores

Un código limpio permitirá al desarrollador no ocultar errores debajo de código repetitivo o difícil de leer, lo que le permitirá ahorrar tiempo y ser más eficiente.

2.5. Productividad del equipo

Todas estas características juntas le permiten al equipo desarrollar el producto más rápido y mantener una calidad constante más fácilmente.

Chapter 3. Reglas de nombrado

Estas reglas establecen las convenciones que el equipo deberá seguir para asignar nombres a los elementos del código, el objetivo es mantener un código **uniforme, claro y fácil de comprender y mantener**. Este proyecto adoptará las convenciones de nomenclatura recomendadas para C# por Microsoft. Solo bajo algunos casos se podrán usar abreviaciones cuyo significado se podrá encontrar al final de la sección.

Se utilizarán principalmente las convenciones *PascalCase* y *camelCase*, más adelante se explicará a más detalle cuando se utilizará cada una. Es importante utilizar en general nombres **significativos** y **descriptivos**(Microsoft, 2026).

3.1. Variables

Las variables deben utilizar nombres acordes con el dominio, claros y descriptivos usando *camelCase*. Se deberán evitar los nombres abreviados o de una sola letra para variables que no sean contadores (ver la sección de "Estructuras de Control" para más información), parámetros de expresiones lambda (ver la sección de "Prácticas de Programación" para ver ejemplos) o convenciones ya conocidas.

El objetivo es que cualquier integrante pueda comprender qué almacena una variable sin tener que revisar su implementación (Microsoft, 2026).

Correcto

```
int playerHealth = 100;  
float movementSpeed = 5.5f;
```

Incorrecto

```
int ph = 100;  
float MOVEMENT_SPEED = 5.5f;
```

3.2. Campos

Los campos de las clases deberán tener nombres descriptivos y ser escritos con **camelCase** (sin importar si son públicos, privados, internos, etc).

Correcto

```
private int currentWeapon;  
private static Random randInstance;
```

Incorrecto

```
private int number_Of_Orders;  
public static Queue csmrQ;
```

3.3. Constantes

Las variables constantes deberán usar **PascalCase**, independientemente de que sean privadas, internas o públicas. Los nombres deberán representar el significado del valor constante (Microsoft, 2026).

Correcto

```
public const int MaximumTry = 3;  
private const double Pi = 3.1459;  
private const float Gravity = 9.81f;
```

Incorrecto

```
private const int a = 10;  
public const double EULER = 2.71828;
```

3.4. Parámetros

Los parámetros de los métodos deben utilizar camelCase y tener nombres que describan claramente el dato que reciben. No se deben usar nombres ambiguos o abreviaturas innecesarias (Microsoft, 2026).

Correcto

```
public void DealDamage(int dealtDamage, int currentHp)  
{  
  
}
```

Incorrecto

```
public void DealDamage(int Dmg, int h)  
{  
  
}
```

3.4.1. Razonamiento

En este caso los parámetros en el segundo ejemplo son incorrectos porque están en PascalCase o

todos en mayúsculas y porque son abreviaciones, por lo que no son descriptivos y no se entiende qué es lo que almacenan.

3.5. Métodos

Los métodos deben tener nombres claros, descriptivos y representar la acción que realizan. Para facilitar su lectura y comprensión, se usará *PascalCase* y no se utilizarán guiones bajos para separar palabras.

Correcto

```
public void StartGame()
{

}

public void CalculatePlayerDamage()
{

}
```

Incorrecto

```
public void start_game()
{

}

public void calcDmg()
{

}
```

3.5.1. Razonamiento

En estos ejemplos, `start_game()` es incorrecto porque no utiliza *PascalCase* y `calcDmg()` es incorrecto porque utiliza una abreviatura poco descriptiva y *camelCase*.

Esta regla busca que la funcionalidad de los métodos puedan identificarse fácilmente por su nombre y que todo el equipo mantenga una convención uniforme.

3.6. Clases

Las clases deberán ser nombradas con un sustantivo descriptivo y escritas con *PascalCase*.

Correcto

```
public class Client
{

}
```

Incorrecto

```
public class myClass
{

}
```

3.6.1. Razonamiento

El nombre myClass no es descriptivo y no está escrito con PascalCase.

3.7. Interfaces

Las interfaces deben comenzar con la letra I mayúscula (de Interface) seguido por el nombre de la interfaz escrito en *PascalCase*. El nombre debe ser descriptivo y representar el comportamiento o contrato que proporciona la interfaz.

Correcto

```
public interface IDamageable
{
    void TakeDamage(int damageAmount);
}
```

Incorrecto

```
public interface Damageable
{
    void TakeDamage(int damageAmount);
}

public interface IL()
{
    void Login();
}
```

3.7.1. Razonamiento

No debe ser necesario leer los métodos en una interfaz para saber qué es o qué hace.

3.8. Enumeraciones

Deberán utilizar PascalCase y se recomienda el uso de sustantivos para nombrarlo.

Correcto

```
public enum WeaponType
{
    Sword,
    Bow,
    Staff
}
```

Incorrecto

```
public enum weapon_types
{
    SWORD,
    BOW,
    STAFF
}
```

3.9. Propiedades

Las propiedades deben ser escritas en PascalCase y deben ser sustantivos que describan claramente el dato o característica que expone la propiedad.

Se utiliza get para obtener valores y set para asignar respectivamente, siempre se escriben en minúsculas debido a que forman parte del lenguaje.

Se pueden crear propiedades personalizadas para agregar lógica adicional al leer o asignar un valor.

Se utilizan las propiedades para:

- Validar datos antes de asignarlos.
- Aplicar reglas de negocio.
- Transformar información antes de devolverla.

No se debe ocupar verbos como GetId o SetAge.

Correcto de propiedades automáticas

```
public int CurrentHealth { get; private set; }
public string PlayerName { get; set; }
```

Correcto de propiedades personalizadas

```
private int name;
public int Name
{
    get => name
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("The name can't be empty");
        name = value;
    }
}
```

Incorrecto de propiedades automáticas

```
public string getName {get; set; }
public string playerName { get; set; }
public string current_health { get; private set; }
```

3.9.1. Cuando usar propiedades automáticas y personalizadas

Se deben usar **propiedades automáticas** cuando:

- No se necesita lógica adicional al obtener o establecer el valor
- Solo se expone un campo de forma simple, sin validación ni transformación
- Se desea mantener el código limpio y conciso — La mayoría de las propiedades en un proyecto bien diseñado deberían ser así
- Se está definiendo DTOs, modelos de datos, o de clases de configuración donde el propósito es solo almacenar datos

Se deben usar **propiedades personalizadas** cuando se necesita: - Validación en el setter (rangos, formatos, valores no nulos, etc.) - Lógica de transformación al asignar o leer (trim, normalización, formateo) - Efectos secundarios como disparar eventos, actualizar otras propiedades, etc. - Lazy loading para calcular o cargar el valor solo la primera vez que se accede - Propiedades calculadas que dependen de otros campos - Control de acceso asimétrico más complejo que private set, como en lógica condicional sobre quién puede modificar el valor

En resumen, por defecto se deben utilizar propiedades automáticas. Se utiliza una propiedad personalizada solo cuando se necesita una lógica adicional.

Si se tienen validaciones repetidas en varias propiedades se recomienda considerar centralizarlas con Data Annotations, un método Validate() o un value object; esto es preferible a llenar una clase de setters personalizados.

3.10. Eventos

Los eventos deben usar PascalCase y sus nombres deberán representar una acción que está ocurriendo o que ya ocurrió. Se recomienda el uso de nombres verbales y distinguir entre acciones previas y posteriores con tiempos verbales como Closing o Closed (Microsoft, 2023).

Buen uso

```
public event EventHandler PlayerDied;  
public event EventHandler GameStarted;
```

Incorrecto

```
public event EventHandler player_died;  
public event EventHandler PlayerDeathEvent;
```

3.11. Manejadores de eventos

Los manejadores de eventos Deberán utilizar PascalCase y se debe usar un guión bajo (_) para separar el nombre del elemento e indicar el activador.

Buen uso

```
private void LoginLink_Click(object sender, RoutedEventArgs e)  
{  
}
```

Incorrecto

```
private void LoginLinkClick(object sender, RoutedEventArgs e)  
{  
}  
  
private void cancel0peration(object sender, RoutedEventArgs e)  
{  
}
```

3.11.1. Razonamiento

PlayerDeathEvent no es un nombre que represente una acción y el sufijo *Event* no es necesario si se tiene un nombre de tipo verbal.

3.12. Delegados

Los tipos delegado debe utilizar PascalCase. Cuando representan manejadores de eventos deben incluir el sufijo EventHandler cuando corresponda (Microsoft, 2026; Microsoft, 2023).

Correcto

```
public delegate void DamageHandler(int damageAmount);
```

Incorrecto

```
public delegate void damage_handler(int damageAmount);
```

3.13. Namespaces

Para los namespaces, los cuales son necesarios para crear claridad para el programador para que cuando se usen sepa perfectamente qué es lo que contiene, se debe usar PascalCase. Pueden usarse nombres en plural cuando sea apropiado, pero no se deberán usarse nombres genéricos como Element, Node, Log y Message. Deberán tener la siguiente estructura:

```
<Proyecto>.<Feature>.<Subnamespace>
```

Correcto

```
namespace MyGame.Combat
{
}
```

Incorrecto

```
namespace mygame.combat_system
{
}
```

3.14. Archivos

Los archivos de código deberán utilizar nombres que identifiquen claramente el tipo o elemento principal que contienen y deberá mantenerse la correspondencia entre el nombre del archivo y el nombre del tipo principal cuando sea aplicable.

Esta regla se establece como una convención del proyecto para facilitar la localización y mantenimiento de código.

Correcto

```
PlayerController.cs  
EnemyController.cs  
InventoryManager.cs  
DamageSystem.cs
```

Incorrecto

```
stuff.cs  
enemies.cs  
code.cs  
clase1.cs
```

3.14.1. Razonamiento

PlayerController.cs guardaría la clase PlayerController, DamageSystem.cs la clase DamageSystem y de esta forma en todos los casos, un ejemplo erróneo sería que stuff.cs guardara la clase PlayerController o que enemies.cs guardara la clase EnemyController.

3.15. Abreviaturas válidas

Las abreviaturas serán válidas en tres casos, que sean referentes a reglas de negocio o dominio o que estén en el siguiente diccionario.

Abreviatura	Significado
DAO	Data Access Object
DB	DataBase
Rand	Random
Max	Maximum
Min	Minimum

3.16. Prefijos y Sufijos

En la capa gráfica se permitirán el uso de prefijos o sufijos, a continuación se nombrarán los prefijos y sufijos utilizados y sus significados, no deberán usarse otros que no estén en esta lista y pueden usarse tanto como sufijos como prefijos para mejorar la legibilidad de la variable.

Sufijo	Significado
Btn	Button
Lbl	Label
Txt	TextBlock, TextBox

Pwd	PasswordBox
Chk	CheckBox
List	ListBox
Img	Image
Stack	StackPanel

Correcto

```
<StackPanel x:Name="InteractionStack" Orientation="Vertical">
    <TextBlock x:Name="TxtTitle" Text = "Super App" />
    <Slider x:Name="UserAge" Minimum="1" Maximum="120" />
    <Label x:Name="AgeLbl" />
    <Button x:Name="PlayBtn" Content="Play" />
    <Button x:Name="QuitBtn" Content="Quit" />
</StackPanel>
```

Incorrecto

```
<StackPanel x:Name="istack" Orientation="Vertical">
    <Label x:Name="lbAge" />
    <Slider x:Name="SAge" Minimum="1" Maximum="110" />
    <Button x:Name="ButtQuit" Content="Quit" />
</StackPanel>
```

Chapter 4. Estilos de código

4.1. Indentación

Para los espacios de indentación deberán usarse espacios en blanco como media, cada nivel equivale a 4 espacios. Un "tab" será igual a 4 espacios en blanco.

Correcto

```
public class Person
{
    private string PrivateName;
    public string Name { get; set; }

    public void PrintName(int repetitions)
    {
        for (int i = 0; i < repetitions; i++)
        {
            Console.WriteLine(name);
        }
    }
}
```

Incorrecto

```
public class Person
{
    private string PrivateName;
    public string Name { get; set; }
}
```

4.2. Longitud máxima de línea

Cada línea debe tener por mucho 120 caracteres, en caso de que se exceda esa cantidad se debe continuar en la siguiente línea cuando se encuentre alguno de los siguientes caracteres:

- Operadores matemáticos (+, -, *, /).
- Paréntesis (), cuando se abren o cierran.
- Comas (,), especialmente en listas de argumentos.
- Llaves ({}), si es el inicio o fin de un bloque de código.
- Corchetes ([]), cuando se declaran o acceden a arreglos.
- Operadores ternarios (?, :)

Correcto

```
string myVeryLongText = "This is a very long text that it's split because it follows  
the standard rools. " +  
    "It's easier to read because the line is not longer than 120 characters.";
```

Incorrecto

```
string myVeryLongText = "This is a very long text that should be split but it's not,  
making it hard to read and maintain in any code editor without horizontal scrolling."
```

4.3. Uso de espacios en blanco

Se usará un espacio en blanco cada vez que escribamos una palabra (el nombre de una variable cuenta como una palabra) o caracter:

Correcto

```
int totalSummatory = firstNumber + secondNumber;  
Console.WriteLine("Calculation result: " + totalSummatory);  
  
for (int i = 0; i < 10; i++)  
{  
    Console.WriteLine("Iteration: " + currentIteration);  
}
```

Incorrecto

```
int sum=a+b;  
Console.WriteLine("Result: "+sum);  
for(int i=0;i<10;i++)  
{  
    Console.WriteLine(i);  
}
```

4.4. Llaves

Las llaves de apertura de clases, interfaces y métodos deben estar una línea después de la línea en la que se hizo la declaración. La llave de cierre debe estar en una nueva línea sola e indentada para coincidir con el comienzo de la declaración.

Los bloques de estructuras de control (while, for, foreach, switch) deben estar encerrados con llaves.

Correcto

```
public class Client()
{
    public void Greet()
    {
        Console.WriteLine("Hello!");
    }
}
```

Incorrecto

```
public class Client() {
    public void Greet() { Console.WriteLine("Hello!"); }
}
```

4.5. Paréntesis

Habrán líneas de código donde el contenido dentro de los paréntesis sea muy largo, la recomendación es usar los paréntesis como llaves, solo que aquí el paréntesis de apertura debe quedarse en la misma línea donde debería empezar.

Correcto

```
Character.Equip(
    fightingEquipment,
    potions,
    fastAbilities
);
```

Incorrecto

```
Character.Equip(fightingEquipment, potions);
Character.Unequip
(
    fightingEquipment,
    potions
);
```

Chapter 5. Organización del código dentro de una clase

Los miembros deberán mantener este orden constante dentro de una clase:

1. Constantes
 - `const float Pi = 3.14159f;`
2. Campos estáticos
 - `static int GameLevel;`
3. Campos de instancia
 - `int Age;`
4. Propiedades
 - `int Weapon { get; set; }`
5. Constructores
6. Métodos públicos
7. Métodos protegidos
8. Métodos privados
9. Eventos

Chapter 6. Prácticas de programación

El uso de las siguientes prácticas es extremadamente recomendado y necesario, se recomienda hacer revisiones de código constantes para validar y verificar su uso:

- Principio de Responsabilidad Única (SRP)
- Evitar código duplicado
- Métodos pequeños y específicos
 - De aproximadamente 30 líneas máximo. Una buena forma de mantenerlos específicos es que hagan únicamente lo necesario para que lleven a cabo la función a la que hace referencia su nombre.
- Número máximo de parámetros
 - Un método no debe tener más de 3 parámetros
- Complejidad ciclomática
 - La complejidad ciclomática de un método no debe superar el valor de 10
- Números mágicos
 - El código no debe tener números mágicos
- Código muerto
 - No hay que encariñarse con el código, el que no se use se debe borrar, **está prohibido comentarlo "para no borrarlo"**
- Valores nulos
 - Se debe evitar usar o regresar valores nulos
- Uso de var
 - El uso de var es recomendado
- Uso de async/await
 - Debe usarse en operaciones asíncronas del videojuego
- Uso de expresiones lambda
 - Se recomienda su uso para hacer que el código sea un poco más corto, limpio y legible.
 - Para más información ver la sección de "Funciones Lambda" más adelante.
- Uso de LINQ
 - LINQ debe usarse siempre que se pueda ya que es una forma de mitigar ciertas vulnerabilidades
 - Para más información ver la sección de "Uso de LINQ" más adelante.
- Optimización
 - Se debe optimizar cuando exista una necesidad justificada y no debe comprometer innecesariamente la legibilidad
- Expresiones regulares

- Se recomienda el uso de expresiones regulares para validaciones

6.1. Funciones Lambda

Son una forma abreviada de escribir una función anónima, es un bloque de código ejecutable que no tiene nombre propio, se puede asignar a una variable de tipo delegado o a una expresión y se pasa usualmente como argumento a otros métodos:

Sintaxis básica

```
(parameters) => expression_or_block
```

Ejemplos:

Con un sólo parámetro:

```
Func<int, int> isEven = n => n % 2 == 0;
```

Con múltiples parámetros:

```
Func<int, int, int> addNumbers = (a, b) => a + b;
```

Sin parámetros:

```
Action greet = () => Console.WriteLine("WAZZAAP!")
```

Con cuerpo de bloque (varias líneas):

```
Func<int, string> classifyNumber = n =>
{
    if (n < 0) return "Negative";
    if (n == 0) return "Zero";
    return "Positive";
};
```

Como argumento directo en LINQ:

```
var player = players.FirstOrDefault(p => p.Name == searchedName);
```

Como Expression<Func<T, bool>> (para proveedores como Entity Framework):

```
Expression<Func<Jugador, bool>> filtro = p => p.Puntos > 100;
```

6.1.1. Nota de nombrado de variables

En las funciones lambda se permite el uso de variables de una sola letra o abreviaciones.

6.1.2. Cuando se deben usar

1. Operaciones LINQ (Where, Select, OrderBy, FirstOrDefault, etc.), donde se expresa una condición o transformación de forma concisa:

```
var playersOnline = players.Where(p => p.online);
```

1. Callbacks o delegados de corta duración, como manejadores de eventos simples o comparadores:

```
matchTable.Sort((a, b) => a.Points.CompareTo(b.Points));
```

1. Inyección de comportamiento en métodos genéricos, como en el patrón Strategy simplificado, esto evita crear una clase o método completo para la lógica de una sola línea.
2. Programación asíncrona ligera, en trabajo puntual como en:

```
Task.Run(() => ...)
```

1. Expresiones que se traducen a otra representación, como en Entity Framework, donde una lambda se convierte en una `Expression<Func<T, bool>>` para generar SQL.

6.1.3. Cuando no se deben usar

1. Cuando la lógica es compleja o extensa. Si supera un aproximado de menos de 10 líneas o necesita varias condiciones anidadas, es mejor extraerla a un método con nombre porque así se mejora la legibilidad y la capacidad de prueba

Correcto

```
var result = players.Where(IsEligiblePlayer).ToList();

bool IsEligiblePlayer(Player p)
{
    var score = p.Wins * 3 + p.Draws;
    var bonus = p.IsCaptain ? 10 : 0;
    var adjusted = score + bonus - p.Penalties;
    return adjusted > 50 && p.YearsActive > 2 && !p.IsSuspended;
}
```

Incorrecto

```
var result = players.Where(p =>
```

```
{
    var score = p.Wins * 3 + p.Draws;
    var bonus = p.IsCaptain ? 10 : 0;
    var adjusted = score + bonus - p.Penalties;
    return adjusted > 50 && p.YearsActive > 2 && !p.IsSuspended;
}).ToList();
```

Razonamiento

Este es un Incorrecto porque la lógica es compleja para una función lambda.

1. Cuando se necesita reutilizar la misma lógica en varios lugares. No se debe copiar y pegar la misma lambda en distintas partes del código.

Correcto

```
Predicate<User> IsAdult = u => u.Age >= 18;
var adults1 = usersA.Where(u => IsAdult(u));
var adults2 = usersB.Where(u => IsAdult(u));
```

Incorrecto

```
var adults1 = usersA.Where(u => u.Age >= 18);
var adults2 = usersB.Where(u => u.Age >= 18);
```

Razonamiento

Se debe definir una vez como un método o predicado para reusarlo, no repetir lambdas.

1. Cuando afecta negativamente el rendimiento en rutas críticas. Las lambdas que capturan variables externas (closures) generan asignaciones de memoria adicionales; en bucles muy exigentes (por ejemplo, dentro de un ciclo ejecutado millones de veces), esto introduce sobrecarga innecesaria.

Correcto

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
{
    int local = i;
    actions.Add(() => AgeLbl.Text = local.ToString());
}
```

Incorrecto

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
```

```
{
    actions.Add(() => AgeLbl.Text = local.ToString());
}
```

Razonamiento

Es incorrecto porque todas las lambdas capturan la misma variable "i", la salida sería 3, 3, 3, no 0, 1, 2.

1. Cuando reduce la depurabilidad. Las lambdas anónimas son más difíciles de inspeccionar en el depurador o de identificar en un stack trace.
2. Cuando el nombre del método aportaría valor semántico. Si una función lambda representa un concepto de negocio importante, es mejor nombrar explícitamente un método.

6.2. Uso de LINQ

LINQ (Language Integrated Query) es un conjunto de características de .NET que permite escribir consultas (filtrar, ordenar, agrupar, transformar) directamente en C#.

Usa una sintaxis unificada, no importa si la fuente de datos es una colección en memoria, una base de datos, XML o cualquier otra fuente que implemente `IEnumerable<T>` o `IQueryable<T>`.

Existen dos estilos de sintaxis equivalentes

Method syntax:

```
var result = players.Where(p => p.Score > 100).OrderBy(p => p.Name);
```

Query Syntax

```
var result = from p in players
              where p.Score > 100
              orderby p.Name
              select p;
```

6.2.1. Cuando se debe usar

1. Filtrar, transformar o agregar colecciones en memoria:

```
var topScores = players.Where(p => p.Score > 100)
                        .OrderByDescending(p => p.Score)
                        .Select(p => p.Name)
                        .ToList();
```

1. Consultas a bases de datos vía un ORM (como Entity Framework)

```
var recentOrders = dbContext.Orders
    .Where(o => o.CreatedAt > DateTime.UtcNow.AddDays(-30))
    .ToList();
```

1. Cuando la legibilidad y el mantenimiento importan más que exprimir cada milisegundo

```
var names = users.Select(u => u.FullName).Distinct();
```

6.2.2. Cuando no se debe usar

1. Rutas de código muy sensibles al rendimiento (hot paths):
 - LINQ añade overhead (delegados, iteradores, a veces asignaciones en el heap) comparado con un bucle for manual

Correcto

```
for (int frame = 0; frame < 1000000; frame++)
{
    for (int i = 0; i < entities.Count; i++)
    {
        if (entities[i].IsVisible)
            Render(entities[i]);
    }
}
```

Incorrecto

```
for (int frame = 0; frame < 1000000; frame++)
{
    var visible = entities.Where(e => e.IsVisible).ToList();
    Render(visible);
}
```

Razonamiento

El uso de ciclos manuales, como en el caso correcto, evita asignaciones y la sobrecarga del iterador en una ruta crítica.

1. Ejecución diferida (deferred execution) sin entenderla, lo que genera bugs o consultas repetidas

Correcto

```
var orders = dbContext.Orders.Where(o => o.Total > 100).ToList();
Console.WriteLine(orders.Count);
```

```
Console.WriteLine(orders);
```

Incorrecto

```
var query = dbContext.Orders.Where(o => o.Total > 100);  
Console.WriteLine(query.Count());  
Console.WriteLine(query.ToList());
```

Razonamiento

En el caso incorrecto la consulta vuelve a ejecutar contra la base de datos cada vez que se enumera.

1. Construir queries con IQueryable mezclando lógica que no se puede traducir a SQL.

Correcto

```
var result = dbContext.Orders  
    .ToList()  
    .Where(o => CustomBusinessLogic(o));
```

Incorrecto

```
var result = dbContext.Orders  
    .Where(o => CustomBusinessLogic(o))  
    .ToList();
```

Razonamiento

En el caso incorrecto Entity Framework no puede traducir un método personalizado de C# a SQL, lo correcto es primero traer la información a la memoria (con AsEnumerable o ToList) y luego aplicar la lógica personalizada.

6.2.3. Vulnerabilidades mitigadas

LINQ no es una herramienta de seguridad explícita, pero indirectamente mitiga varias vulnerabilidades al reemplazar patrones manuales riesgosos como:

- Inyección SQL—Aplica cuando se usa LINQ to Entities/Entity Framework correctamente, si se mezcla LINQ con FromSqlRaw o ExecuteSqlRaw y se concatenan strings la vulnerabilidad vuelve.
- Errores de manejo manual de índices o bucles (off-by-one, null references)
- Exposición de datos sensibles por sobre-selección (over-fetching)

Chapter 7. Estructuras de control

7.1. if - else

Debe haber un espacio entre **if** y la condición, se deben utilizar llaves si el resultado es de más de una línea.

El **else** debe ir en la línea de abajo de la llave de cierre del **if**.

Correcto

```
public void CheckAge(int studentAge)
{
    int adultStudents = 0;
    int underageStudents = 0;
    int adultAge = 18;
    if (studentAge >= adultAge)
    {
        adultStudents++;
        Console.WriteLine("The student is an adult.");
    }
    else
    {
        underageStudents++;
        Console.WriteLine("The student is underage.");
    }
}
```

Incorrecto

```
public void CheckAge(int studentAge)
{
    int adultStudents = 0;
    int underageStudents = 0;
    int adultAge = 18;
    if(studentAge >= adultAge) {
        adultStudents++;
        Console.WriteLine("The student is an adult.");
    } else { underageStudents++; Console.WriteLine("The student is underage."); }
}
```

7.2. If de una línea

Se permite utilizar el **If** de una sola para funciones lambda. Para métodos debe llevar corchetes.

Correcto

```
Func<int, string> classifyNumber = n =>
{
    if (n < 0) return "Negative";
    if (n == 0) return "Zero";
    return "Positive";
};
```

Incorrecto

```
public void CheckAge(int studentAge)
{
    int adultAge = 18;
    if (studentAge >= adultAge) Console.WriteLine("The student is an adult.");
    else
        Console.WriteLine("The student is underage.");
}
```

7.3. Else If

La estructura tiene que comenzar en la línea debajo de la última llave del bloque **if**. La estructura **else if** tiene que ir seguida por un espacio en blanco antes de la condición y la llave de inicio.

Correcto

```
public void CheckGrade(int grade)
{
    int excellentGrade = 8;
    int goodGrade = 6;
    if (grade >= excellentGrade)
    {
        GradeLbl.Text = "Excellent!";
    }
    else if (grade >= goodGrade)
    {
        GradeLbl.Text = "Good!";
    }
    else
    {
        GradeLbl.Text = "You can do it better!";
    }
}
```

Incorrecto

```
public void CheckGrade(int grade)
{
```

```

int excellentGrade = 8;
int goodGrade = 6;
if (grade >= excellentGrade)
{
    GradeLbl.Text = "Excellent!";
}
else
if (grade >= goodGrade)
{
    GradeLbl.Text = "Good!";
}
else
{
    GradeLbl.Text = "You can do it better!";
}
}

```

7.4. Switch

Debe declararse en una nueva línea y debe tener un espacio antes de la condición. Cada uno de los **case** deben ser indentados un nivel.

Correcto

```

public void Menu(int option)
{
    switch (option)
    {
        case 1:
            Console.WriteLine("You select the option 1");
            break;
        default:
            Console.WriteLine("Select a valid option");
            break;
    }
}

```

Incorrecto

```

public void Menu(int option)
{
    switch(option)
    {
        case 1:
            Console.WriteLine("You select the option 1");
            break;
        default:
            Console.WriteLine("Select a valid option");
    }
}

```

```
    break;
  }
}
```

7.5. Operadores ternarios

El uso de operadores ternarios está permitido si se cumplen las siguientes condiciones:

- Solo hay dos posibles valores para asignar a una variable
- Otra persona puede entender la lógica detrás del operador a la primera leída
- No se permite la anidación de operadores ternarios bajo ninguna circunstancia

Correcto

```
int adultAge = 18;
int adultAccessFee = 0;
int minorsAccessFee = 10;
var isAdult = age >= adultAge;
var accessFee = isAdult ? 0 : 10;
```

Incorrecto

```
var triangleType = (sideA == sideB && sideB == sideC) ? "Equilateral" :
    (sideA == sideC || sideB == sideC || sideA == sideC) ?
    "Isosceles" : "Scalene";
```

7.6. While

Debe existir un espacio entre la palabra **while** y la condición. Si el bloque solo contiene una línea está permitido el omitir los corchetes.

Correcto

```
while (numberA >= 0)
{
    Console.WriteLine(numberA);
}

while (numberA >= 0)
    Console.WriteLine(numberA);
```

Incorrecto

```
while (numberA >= 0) {
```

```
    Console.WriteLine(numberA);  
}  
  
while(true) Console.WriteLine("Executing...");
```

7.7. Do-While

En la línea siguiente a la palabra **do** debe ir la llave de inicio, la palabra **while** debe ir en la misma línea de la llave de cierre, separado por un espacio de la condición.

Correcto

```
int numberA = 0;  
do  
{  
    Console.WriteLine(numberA);  
    i++;  
} while (numberA <= 5);
```

Incorrecto

```
string input;  
do {  
    input = Console.ReadLine();  
} while (!CheckInput(input));
```

7.8. For

Debe haber un espacio entre la palabra **for** y la condición, la declaración dentro de los paréntesis debe estar separada por **;** junto con un espacio después de cada uno, la llave de inicio deberá ir en la línea siguiente a la declaración del **for**.

En este caso, por ser una práctica muy común se permite que la variable del contador sea una letra como: **i**, **j**, **k**.

Correcto

```
int evenCounter = 0;  
for (int currentNumber = 1; currentNumber <= 20; currentNumber++)  
{  
    if (currentNumber % 2 == 0)  
    {  
        evenCounter++;  
    }  
}
```

```
Console.WriteLine("Even numbers:" + evenCounter);
```

Incorrecto

```
int sum = 0;
for(int currentNumber = 0;currentNumber<10;currentNumber++)
{
    sum += currentNumber;
}
Console.WriteLine("Sum:" + sum);
```

Chapter 8. Manejo de Excepciones

Las excepciones se deben propagar cuando los métodos donde puede ocurrir la excepción se llamarán desde otro método de otra clase, y en el caso donde se llaman es donde se capturarán las excepciones lanzadas. Un ejemplo de esto es un DAO que puede lanzar un `SQLException`, dentro del DAO se propagan mientras que capturamos las excepciones cuando los mandamos a llamar.

En el caso de que un cliente llame a un método de un servidor el servidor registra en su log la excepción y le envía al cliente un *`FaultException`* con la información que le interesa.

Todos los errores se deben registrar en la bitácora (logger). En caso de que un error cause el cierre total del programa se debe informar al usuario que hubo un error inesperado y que se cerrará la aplicación, en casos como estos es de suma importancia que se registre en la bitácora. Se debe intentar perder la menor cantidad de información posible.

Siempre que ocurra un error se debe informar a los interesados, por ejemplo, si no se pudo escribir alguna actualización solicitada por el usuario se le deberá avisar mediante un mensaje en la pantalla, si hubo algún error de compilación o vulnerabilidad que no afecte directamente a la UX se deberá registrar en el logger.

Las excepciones de tipo `Exception` se usarán como último recurso en el nivel de cacheo, es decir, primero se atraparán las excepciones específicas y por último la de tipo `Exception` en caso de ser necesario. Este manejo debe estar muy bien vigilado y justificado porque puede resultar en un antipatrón.

8.1. Bloques Try-Catch

Dentro del **try**, de preferencia, solo deberá estar el código que pueda lanzar una excepción, y la variable que almacena la excepción atrapada deberá llevar el nombre *ex* o *e*, debe usarse la misma en cada bloque try-catch.

Las llaves deberán estar abajo de la declaración del **try** y **catch**.

Se evitará el uso de atrapar *Exception* genérica y se recomienda el uso de excepciones específicas.

Correcto

```
try
{
    int[] numbers = {1, 2, 3};
    Console.WriteLine(numbers[5]);
}
catch (IndexOutOfRangeException ex)
{
    logger.error("NumberArray.PrintNumber" + ex);
}
```

Nota: Este fragmento siempre lanzará excepción, solo es un ejemplo.

Incorrecto

```
try {  
    int[] numbers = {1, 2, 3};  
    Console.WriteLine(numbers[5]);  
} catch (SQLException ex) {  
    Console.WriteLine(ex.Message);  
} catch (Exception e) {  
    Console.WriteLine(e.Message);  
}
```

¡Si estás atrapando "Exception ex" significa que no sabes realmente qué puede fallar en tu código!

Chapter 9. Registro y depuración (bitácora)

Se debe usar **log4Net** para darle seguimiento a los sucesos de la aplicación (cuando inicia el servidor, se crea un juego, etc.), advertencias, errores, errores fatales y excepciones, se deberá mostrar el log en consola y en un archivo .log. Es importante respetar las jerarquías de las bitácoras (info, warning, error, fatal).

Correcto

```
private static readonly ILog logger = LogManager.GetLogger(typeof(CoolVideogame));

private static void Main(String[] args)
{
    logger.info("The server is online");

    try
    {
        CoolVideogame.Setup().ConnectToDatabase();
    }
    catch (TimeoutException ex)
    {
        logger.info(ex);
    }
    catch (SQLException ex)
    {
        logger.error(ex);
    }
    catch (FileNotFoundException ex)
    {
        logger.fatal(ex);
    }
}
```

Incorrecto

```
private static readonly ILog logger = LogManager.GetLogger(typeof(CoolVideogame));

private static void Main(String[] args)
{
    try
    {
        CoolVideogame.Setup().ConnectToDatabase();
    }
    catch (SQLException e)
    {
        logger.error("Error reading something", e);
    }
    catch (TimeoutException ex)
```

```
{
    logger.fatal(ex);
}
catch (Exception e)
{
    logger.info(e);
}
}
```

9.1. Identificación de tipos de error

9.1.1. Fatales

Se consideran fatales las excepciones de las que la aplicación no podrá recuperarse y deberá cerrarse, por ejemplo:

- `OutOfMemoryException`
- `StackOverflowException`
- `FileNotFoundException` (de archivos críticos de la aplicación)

9.1.2. Errores

Se consideran errores las excepciones de las cuales sí se puede recuperar la aplicación, por ejemplo:

- `IOException`
- `NullPointerException`
- `EntityException`

9.1.3. Advertencias

Se consideran advertencias las excepciones que sean momentáneas, es decir, que por alguna razón extraordinaria sucedieron y si se vuelve a ejecutar el proceso no deberían volver a pasar. También entran en esta categoría las que no afecten el rendimiento y/o funcionamiento de la aplicación. Ejemplos:

- `TimeoutException`
- `ResourceNotFoundException`
- `FileNotFoundException` (En el caso de archivos que no afectan el funcionamiento del sistema, como fotos de perfil)

9.1.4. Info

Se considera info a momentos claves en la aplicación que es útil saber cuando suceden, por ejemplo: cuando se inicia la aplicación, se conecta la base de datos, etc.

Chapter 10. Comentarios

El uso de comentarios solo será permitido cuando la implementación que se dio es fuera de lo normal o requiere una explicación más avanzada y para anotar banderas de desarrollo.

10.1. Comentarios de línea única

Los comentarios de línea única deben estar rodeados de una línea en blanco, se usarán para expresar:

- **TO-DO**: Funcionalidad que debe codificarse
- **FIX**: Se debe arreglar alguna parte de ese código o funcionalidad
- **TEST**: Se necesita probar esa funcionalidad
- **REFACTOR**: Se debe refactorizar esa funcionalidad

El mensaje debe estar entre dobles diagonales, un espacio y tres asteriscos:

```
// *** TO-DO *** //
```

Correcto

```
// *** FIX *** //

public void FibonacciSequence()
{
}

// *** TO-DO *** //

public void EulerSequence()
{
}

// *** REFACTOR *** //

public async Task GetAllStudents()
{
}
```

Donde *GetAllStudents()* es un método considerado grande (una buena referencia es que tenga más de 30 líneas), viola el principio de responsabilidad única o tiene una complejidad ciclomática de más de 10, por poner algunos ejemplos.

Incorrecto

```
// This method does this x
public void SuperCalculator()
{
}

public void foo(List<T> l)
{
    // iterate over list
    foreach (var something in l)
    {
        // process something
    }
}
```

10.2. Comentarios de bloque

Los comentarios de bloque se pueden ocupar (no son recomendados) únicamente para explicar una implementación extraña o poco común en algún método ya probado, por ejemplo:

- Implementaciones que no sean entendibles en una primera lectura.
- Soluciones que no siguen un patrón ya existente para ese problema. — Se debe explicar por qué no se usó la solución ya existente y probada.

En general, los comentarios jamás deben explicar qué hace el código, sino el por qué se escribió así (en caso de que requiera explicación).

Para comentar en bloque se usarán los comentarios multi línea (`/*, */`), se abrirá el comentario y se comenzará a escribir el comentario en la línea siguiente y se cerrará el comentario una línea después de que termine el mensaje.

Correcto

```
/*
ShuffleCards is built this way because...
*/

public void ShuffleCards(List<Card> cards)
{
}
```

Incorrecto

```
/* This method doesn't work well
*/

/*
This method is for...
```

*/

En el segundo ejemplo del Incorrecto es incorrecto porque la funcionalidad de un método debe poderse deducir por su nombre, por lo que un comentario no debe explicar el qué hace.

10.3. Comentarios de una línea en XAML

Los comentarios de una línea en XAML se permitirán para explicar la sección o el contenido que se está describiendo, son más como divisores para entener el orden y ubicar más rápido lo que se quiera editar.

Correcto

```
<Grid>
<!-- Column 1 -->

<Label x:Name="PageTitleLbl" Content="{x:Static p:Resources.DefaultTitle}"
      Grid.Column="1" />

<!--Column 2 -->
<Button x:Name="SubmitPageBtn" Content="{x:Static p:Resources.Submit}"
      Grid.Column="2" />

</Grid>
```

Incorrecto

```
<Grid>
<!-- This is the page title, that will dinamically change ... -->

<Label x:Name="PageTitleLbl" Content="{x:Static p:Resources.DefaultTitle}"
      Grid.Column="1" />

<!-- This column will be doing the submit button which will work ... -->
<Button x:Name="SubmitPageBtn" Content="{x:Static p:Resources.Submit}"
      Grid.Column="2" />

</Grid>
```

10.4. Comentarios de API pública

Conocidos como *Comentarios de Documentación*, se escribe con tres diagonales (///), es recomendable que incluya un summary mediante una etiqueta XML (<summary></summary>) donde se escribe un resumen del método y las etiquetas que correspondan, las más comunes son:

- <param name="..."> Explica cada parámetro.

- `<returns>` Describe qué devuelve la función.
- `<remarks>` Agrega notas adicionales (a mayor detalle).
- `<example>` Muestra un ejemplo de uso.
- `<exception>` Describe la excepcion que la API puede tirar y bajo qué circunstancias.

10.4.1. Ejemplo

```

/// <summary>
/// Represents a basic calculator
/// </summary>
public class Calculator
{
    /// <summary>
    /// Subtraction of two numbers
    /// </summary>
    /// <param name="firstNumber"> The number we are subtracting of</param>
    /// <param name="secondNumber"> The number we are subtracting to firstNumber
    </param>
    /// <returns> The remainder of firstNumber - secondNumber </returns>
    /// <example>
    /// Example of use:
    /// <code>
    /// var calculator = new Calculator();
    /// int result = calculator.Substraction(10, 3);
    /// </code>
    /// </example>

    public int Subtraction(int firstNumber, int secondNumber)
    {
        return firstNumber - secondNumber;
    }
}

```

Chapter 11. Seguridad

Desarrollar un sistema seguro es de gran importancia, especialmente si será un sistema con conexión en red, por lo que mencionamos aquí las principales pautas a seguir para poder mantener la información íntegra y privada.

_ Validación de entradas. - Las contraseñas de los usuarios deberán estar hasheadas. - Las credenciales del sistema para ingresar a bases de datos o algún otro sistema deberán guardarse en otro archivo de configuración al que se accederá desde el código. **Una opción es usar variables de entorno.** - **Jamás debe haber credenciales en el código.** - **El código deberá estar libre de smells y no deberá ser susceptible a inyecciones, esto se puede revisar con alguna herramienta de revisión de código como SonarQube.** - **Las contraseñas deberán tener una longitud mínima de 8 caracteres y al menos un caracter de los siguientes:** Mayúsculas **Minúsculas** Número ****** Caracter especial (@-_/) - Los usuarios deben tener privilegios personalizados y limitados.

Chapter 12. Pruebas

Las pruebas deberán abarcar algunos de los siguientes casos si es que aplican:

1. Caso ideal
2. Entradas límite
3. Entradas inválidas
4. Comportamiento con datos duplicados
5. Efectos colaterales
6. Excepciones
7. Comprobación de retorno

Solo deberán contar con una aserción por prueba.

12.1. Nombrado de pruebas

El nombre de una prueba consta de tres partes, la primera es el prefijo Test, seguido por el método, flujo o comportamiento que se probará y la última parte será el camino que se está calificando (exitoso, entradas mínimas, entradas vacías, retorno, excepción etc.).

Correcto

```
public void TestLoginSuccess()  
public void TestDatabaseConnectionWrongPassword()  
public void TestPlayersSyncInvalidToken()  
public void TestPlayerUpdateNullException()
```

Incorrecto

```
public void RegisterTaskTest()  
public void CalculateSalaryWrong()  
public void TestConnectToDatabaseShouldReturnTrue()
```

Chapter 13. Control de versiones

El control de versiones en la producción de proyectos actual es básica y de suma importancia puesto que permite que el equipo mantenga un historial organizado de los cambios realizados al videojuego, trabaje de forma paralela y revise las modificaciones antes de incorporarlas a las ramas principales. Es obligatorio seguir las siguientes reglas.

13.1. Sistema de control de versiones

Se usará Git como sistema de control de versiones y todos los cambios relevantes del código fuente deben registrarse mediante *commits* para que exista un historial que permita identificar cómo ha evolucionado el proyecto.

13.2. Repositorio

Debe haber un repositorio por cada proyecto de .NET que tenga el producto y debe ser compartido por todos los integrantes del equipo.

Git permite trabajar con repositorios compartidos y ramas remotas, mientras que GitHub proporciona mecanismos para colaborar sobre el repositorio mediante ramas y Pull Requests (Chacon & Straub, 2014).

Cada repositorio debe contener el código y los archivos necesarios para desarrollar y construir el videojuego, evitando incorporar archivos innecesarios o generados automáticamente.

Correcto

```
Repositorio
├── Assets/
├── Scripts/
├── Scenes/
├── Tests/
├── README.md
└── .gitignore
```

Incorrecto

```
Repositorio
├── Assets/
├── Scripts/
├── build-final.zip
├── screenshots/
├── ProyectoViejo/
├── backup/
└── cosas/
```

13.2.1. Razonamiento

El ejemplo incorrecto mezcla archivos del proyecto con respaldos, versiones antiguas y archivos que no forman parte del desarrollo.

13.3. Estructura de ramas

Las ramas se utilizarán para aislar el desarrollo de funcionalidades o correcciones y evitar que el trabajo afecte directamente a la rama principal.

Se establece la siguiente estructura mínima para cada rama:

```
main
|
|- feature/*
|- fix/*
|- exp/*
|- hotfix/*
```

Donde: - main: Versión estable del proyecto - feature/: **Desarrollo de nuevas funcionalidades** - **fix**/: Corrección de errores - exp/: **Para cuando se desee probar soluciones fuera de lo común o experimentar** - **hotfix**/: Correcciones urgentes sobre una versión estable

Correcto

```
feature/player-movement
feature/inventory-system
fix/player-collision
hotfix/save-game-error
exp/out-of-box-player-hitbox-algorithm
```

Incorrecto

```
gabosaurio
test
new-things
user-branch
```

13.3.1. Razonamiento

Los nombres incorrectos no comunican el propósito de la rama y dificultan que los demás integrantes entiendan qué trabajo contiene.

13.4. Commits

Cada commit debe indicar con uno de los siguientes prefijos:

- feat: Para indicar una nueva funcionalidad
- fix: Para indicar que se arregló algo
- refactor: Para indicar que se refactorizó alguna parte del proyecto
- conflicts: Para indicar que se resolvieron conflictos con alguna rama
- config: Para archivos de configuración
- docs: Para indicar que se actualizó la documentación

Esto se debe acompañar de un mensaje breve y descriptivo. El mensaje deberá permitir comprender qué se modificó sin tener que inspeccionar inmediatamente el código.

Correcto

```
git commit -m "feat: basic player movement implemented"
git commit -m "fix: images bug that the profile page showed when opened"
git commit -m "refactor: to the profile image downloads algorithm"
git commit -m "conflicts: solving conflicts with main"
git commit -m "config: gitignore updated"
git commit -m "docs: readme updated"
```

Incorrecto

```
git commit -m "changes"
git commit -m "feat"
git commit -m "basic player turn implemented"
git commit -m "lajdf"
```

13.4.1. Razonamiento

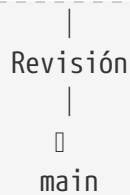
Los commits no pueden solo tener una palabra ni únicamente el tipo de cambio que se hizo. Tampoco puede solo tener el mensaje porque no indica el tipo de cambio. Finalmente, el mensaje del commit no pueden solo ser caracteres aleatorios.

13.5. Pull Requests

Los cambios destinados a incorporarse a la rama principal deben hacerse mediante un *Pull Request*, en lugar de modificarla directamente. Un *Pull Request* permite proponer cambios desde una rama, revisarlos e integrarlos posteriormente, este mecanismo ayuda a detectar problemas antes de que lleguen a la rama principal y ayuda a evitar conflictos.

Correcto

```
feature/inventory-system
|
□
Pull Request
```



Incorrecto

```
git switch main
git add .
git commit -m "Agregar inventario"
git push
```

13.5.1. Razonamiento

Este ejemplo es incorrecto porque se está desarrollando directamente en la rama principal (main), por lo que se viola el flujo de desarrollo definido en este estándar.

13.6. Resolución de conflictos

Cuando existan cambios incompatibles que generen conflictos entre ramos se deberán resolverse antes de realizar el *merge*. El desarrollador responsable debe revisar manualmente las diferencias, conservarla implementación correcta y verificar posteriormente que el proyecto continúe funcionando.

Una vez que los conflictos se resuelven deben realizarse las pruebas correspondientes para corroborar que el funcionamiento el que se esperaba, es correcto y no se vio afectado.

Chapter 14. Declaración de IA

Se declara el uso de IA para verificar la gramática del estándar, generación de ejemplos, apoyo en la estructuración del documento y comprensión de algunos temas.

14.1. Prompts

Hola, toma este archivo como referencia, no lo tomes como referencia únicamente como estructura. Tengo que realizar mi estándar de desarrollo para mi proyecto que será un videojuego, el lenguaje de programación que se usará va a ser C#. Genera la estructura del archivo de referencia y número las secciones. También dame la cadena de pensamiento auditable que hiciste por favor.

No tomes como referencia de consulta el archivo que te pasé.
Las referencias de consulta deben ser el estándar oficial de C# o reportes oficiales de Microsoft o Estándares de código validados y de empresas verificables o documentación oficial del lenguaje.
Genera una descripción corta de la sección 3.7 que sirva para que todos los integrantes del equipo entiendan el objetivo de la regla, el estándar es para un proyecto escolar.
Incluye un ejemplo que sea correcto de acuerdo a las convenciones del estándar y a la sección y que incluya un ejemplo que viole el estándar.
El texto de la sección debe estar sustentado en una fuente verificable e incluye la cita en APA versión 7 en el texto y también genera la referencia en APA 7 para la sección de referencias del estándar.
Genera la cadena de razonamiento auditable para verificar el contenido.

No tomes como referencia de consulta el archivo que te pasé.
Las referencias de consulta deben ser el estándar oficial de C# o reportes oficiales de Microsoft o Estándares de código validados y de empresas verificables o documentación oficial del lenguaje.
Genera una descripción corta de las secciones 3.1 a la 3.15 que sirva para que todos los integrantes del equipo entiendan el objetivo de la regla, el estándar es para un proyecto escolar.
Incluye un ejemplo que sea correcto de acuerdo a las convenciones del estándar y a la sección y que incluya un ejemplo que viole el estándar.
El texto de la sección debe estar sustentado en una fuente verificable e incluye la cita en APA versión 7 en el texto y también genera la referencia en APA 7 para la sección de referencias del estándar.
Genera la cadena de razonamiento auditable para verificar el contenido.

Me puedes dar una pequeña definición de las funciones lambda, cuando es buena práctica utilizarlas, cuando no se deben utilizar y como se utilizan. Todo esto en un contexto de desarrollo para .NET
También incluye ejemplos correctos e incorrectos en inglés para cada caso por favor.

Al final genera tu cadena de razonamiento

Me puedes dar una pequeña definición de LINQ, cuando es buena práctica utilizarlo, cuando no se debe utilizar, como se utiliza, para qué sirve y qué vulnerabilidades mitiga.

Todo esto en un contexto de desarrollo para .NET También incluye ejemplos correctos e incorrectos en inglés para cada caso por favor.

Al final genera tu cadena de razonamiento

Chapter 15. Bibliografía

BillWagner. (n.d.-a). Identifier names - rules and conventions - C#. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>

BillWagner. (n.d.-b). .NET Coding Conventions - C#. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Chidera, I. (2023, January 13). C# Coding Best Practices – Coding Conventions with Examples. freeCodeCamp.org. <https://www.freecodecamp.org/news/coding-best-practices-in-c-sharp/>

De Tinchicus, V. T. L. E. (2020, October 27). C# / Getter and setter. El Blog De Tinchicus. <https://tinchicus.com/2020/10/27/c-getter-and-setter/>

BillWagner. (n.d.). Documentation comments - C# language specification. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/documentation-comments>

KrzysztofCwalina. (n.d.). Names of Namespaces - Framework design Guidelines. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-namespaces>

BillWagner. (n.d.-b). Instrucciones de control de excepciones: throw y try, catch, finally - C# reference. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/statements/exception-handling-statements>

Apache log4net. (n.d.-b). <https://logging.apache.org/log4net/manual/introduction.html>

Microsoft. (2026). Identifier names - rules and conventions - C#. Microsoft Learn. Microsoft Learn — Identifier names. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>

Microsoft. (2023, October 3). Capitalization conventions. Microsoft Learn. Microsoft Learn — Capitalization conventions. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/capitalization-conventions>

Microsoft. (2026). Names of classes, structs, and interfaces. Microsoft Learn. Microsoft Learn — Names of classes, structs, and interfaces. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-classes-structs-and-interfaces>

Microsoft. (2026). Names of type members. Microsoft Learn. Microsoft Learn — Names of type members. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-type-members>

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2/>

Git. (2026). Git user manual. <https://git-scm.com/docs/user-manual>

Git. (2026). git-commit documentation. <https://git-scm.com/docs/git-commit>

GitHub. (2026). About protected branches. GitHub Docs. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

GitHub. (2026). About pull request reviews. GitHub Docs. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews>

GitHub. (2026). About pull requests. GitHub Docs. <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>

PhoenixNAP. (n.d.). Git branch naming convention. <https://phoenixnap.com/kb/git-branch-name-convention>

Conventional Commits. (n.d.). Conventional Commits 1.0.0. <https://www.conventionalcommits.org/en/v1.0.0/>